CDAC Delhi

A

Project Report on

"C&D — WHERE CLIENT AND DEVELOPER MEET"

Submitted in partial fulfillment of the requirements of
Post Graduate Diploma in Advanced Computing (PG-DAC)

Submitted By
February 2026

Mr. Akshat Shokeen (250810120005)
Mr. Abuzer Amaan (250810120003)
Mr. Kumar Apoorb (250810120007)

GUIDE BY

Prof. Pankaj Kumar Mahto
Faculty, CDAC Delhi
Centre for Development of Advanced Computing Delhi

CERTIFICATE

This is to certify that the Project Report entitled

"C&D — WHERE CLIENT AND DEVELOPER MEET"

Has been duly completed by the following students under guidance, in a satisfactory manner as a partial fulfillment of the requirements for the award of Post Graduate Diploma in Advanced Computing (PG-DAC):

Submitted By

Mr. Akshat Shokeen (250810120005)
Mr. Abuzer Amaan (250810120003)
Mr. Kumar Apoorb (250810120007)

Prof. Pankaj Kumar Mahto
Project Guide, Faculty CDAC Delhi

Centre Director
CDAC Delhi

DECLARATION

We hereby declare that the project entitled "C&D — Where Client and Developer Meet" submitted for the award of Post Graduate Diploma in Advanced Computing (PG-DAC) is our original work. The content presented in this report represents our ideas in our own words; where others' ideas have been included, we have adequately cited the original sources.

We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, or source in our submission. Any violation of the above will be cause for disciplinary action by the Institute.

Date: 18/02/2026

Mr. Akshat Shokeen Mr. Kumar Apoorb Mr. Abuzer Amaan

ACKNOWLEDGEMENT

We are immensely thankful to Prof. Pankaj Kumar Mahto, Faculty at CDAC Delhi, for his patient guidance, constructive suggestions, and consistent encouragement throughout the duration of this project. His mentorship gave us the confidence to tackle a complex two-sided platform and bring it to a working state.

We extend our sincere gratitude to the entire CDAC Delhi department — faculty, technical staff, and administrative staff — for their cooperation.

We also thank our batchmates and friends who made the development journey collaborative and enjoyable. The exchange of ideas and peer feedback were invaluable in shaping the final product.

Lastly, we express our deepest gratitude to the Almighty for blessing us with the strength and perseverance to complete this project successfully.

— PROJECT TEAM

ABSTRACT

C&D (Client & Developer) is a two-sided web platform that connects clients who need software built with skilled developers who can build it. Clients can browse developer profiles, view portfolios, send project inquiries, and chat in real time. Developers can create and edit their profiles, showcase portfolio projects, manage incoming leads, and communicate with clients through a built-in chat interface.

The platform is built using Spring Boot 3.2 (Java 21), Hibernate/JPA, MySQL, Thymeleaf, and a custom CSS/JavaScript frontend. The backend exposes a clean set of REST APIs covering authentication (signup/login), developer management, client profile management, project inquiries, and a full chat messaging system (threads and messages). All data is persisted in a MySQL database with a well-normalised schema across eight tables.

The project was developed in two phases: a UI-first demo phase using mock data, followed by a full Spring Boot backend integration phase that replaced mock services with real API calls. The result is a functionally complete, role-based web application that demonstrates end-to-end full-stack Java development skills acquired during the PG-DAC program at CDAC Delhi.

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Overview

The freelance and contract software development market is growing rapidly. Businesses of all sizes increasingly need software products — websites, web apps, MVPs, and internal tools — but lack in-house technical talent. At the same time, skilled developers struggle to find quality leads and showcase their work to the right clients.

C&D (Client & Developer) is a two-sided marketplace platform that bridges this gap. It provides a clean, modern web interface where clients can discover developers, review their portfolios, send project inquiries, and communicate through a built-in messaging system — all in one place. Developers, on the other hand, can build a public profile, manage their portfolio of past work, and engage with incoming client leads.

This project was developed as part of the PG-DAC (Post Graduate Diploma in Advanced Computing) program at CDAC Delhi, and demonstrates mastery of full-stack Java development using Spring Boot, Hibernate/JPA, MySQL, Thymeleaf, and modern frontend techniques.

1.2 Background & Motivation

Platforms such as Upwork and Fiverr serve global markets but are often noisy, commoditised, and charge high commissions. C&D is designed as a clean, premium-feel alternative that can be deployed by an institution, accelerator, or talent network to connect its own community of clients and developers.

The project was conceived to provide students with end-to-end experience in building a real-world, multi-role web application covering authentication, relational data modelling, REST API design, and frontend-backend integration.

1.3 Scope

The platform covers the following functional areas:

- User registration and login with role-based routing (CLIENT / DEVELOPER).
- Developer profile creation, editing, and public listing with skill and availability data.
- Portfolio project management — add and showcase past work with tech stack details.
- Client profile management — company name, industry, and budget range.
- Inquiry submission — clients send project summaries to developers.
- Real-time-style chat — threaded messaging between clients and developers.
- Responsive, premium UI served via Thymeleaf templates and custom CSS/JS.

Chapter 2: Problem Statement & Objectives

2.1 Problem Statement

The following pain points exist in the current freelance software hiring landscape and motivated the design of C&D:

- Clients have no curated, trustworthy space to discover developers with verified portfolios and transparent rates.
- Developers have no simple way to present themselves professionally without building a personal website from scratch.
- Existing platforms lack a focused, premium-feel experience for the serious, project-based hiring segment.
- Communication between clients and developers is scattered across emails and external tools, with no unified thread.
- There is no lightweight, self-hostable alternative for institutions, cohorts, or communities to run their own talent marketplace.

2.2 Objectives

The primary objectives of the C&D platform are:

- Build a two-sided web application supporting distinct Client and Developer user flows.
- Implement secure user registration and login with role-based access.
- Enable developers to create rich profiles with skills, hourly rate, availability, and portfolio projects.
- Enable clients to browse and filter developers, view profiles, and send project inquiries.
- Implement a threaded in-platform messaging system for client-developer communication.
- Expose all core functionality via clean REST APIs ready for future mobile or SPA frontends.
- Persist all data in a well-normalised MySQL schema with Hibernate/JPA integration.
- Deliver a polished, responsive UI with a premium tech-marketplace aesthetic.

2.3 Non-Goals (Current Phase)

The following are intentionally out of scope for the current version:

- Payment gateway integration — clients and developers handle payments personally.
- Production-grade security (Spring Security, JWT, OAuth2) — basic email/password authentication only.
- File upload for resumes or portfolio images — URLs are used as placeholders.
- Mobile native application — responsive web only.

Chapter 3: System Architecture

3.1 High-Level Architecture

C&D follows a layered MVC architecture served by a single Spring Boot application. The frontend is composed of Thymeleaf HTML templates enhanced by vanilla JavaScript that calls the backend REST APIs. The backend exposes REST endpoints which are consumed both by the Thymeleaf pages (via fetch()) and can be consumed by any future SPA or mobile client.

Layer	Description
Presentation	Thymeleaf HTML templates + custom CSS (styles.css) + JavaScript (app.js, data.js). Renders pages served at named routes. JS calls /api/* endpoints via fetch() and falls back to mock data (data.js) if API is unavailable.
Page Controller	PageController.java — maps named URL routes (/, /login, /client/home, /developer/home, etc.) to Thymeleaf template names. No business logic.
REST API Layer	Six @RestController classes under com.cd.platform.api. Handle request parsing, validation, and DTO mapping. Call service methods.
Service Layer	Five @Service classes. Contain all business logic: authentication, developer/client CRUD, portfolio management, inquiry handling, and chat orchestration.
Repository Layer	Eight JpaRepository interfaces. Auto-generate SQL queries via Spring Data JPA. No manual SQL required.
Model / Entity Layer	Eight @Entity classes mapped to MySQL tables via Hibernate.
Database	MySQL 8.0. Schema: cd_platform. Auto-validated by Hibernate (ddl-auto=validate). Seeded via db/seed.sql.

3.2 User Roles & Flows

The platform has two user roles. Role is stored on the User entity and determines the post-login redirect and accessible pages.

Role	Flow
CLIENT	Login → /client/home (dashboard) → /client/developers (browse) → /client/developer?id=X (profile + inquiry) → /client/messages (chat)
DEVELOPER	Login → /developer/home (dashboard + leads) → /developer/profile (edit profile) → /developer/portfolio (manage projects) → /developer/messages (chat)

3.3 Project Folder Structure

```
v3 (Dual)/
├── pom.xml                                  ← Maven build & dependencies
├── db/seed.sql                              ← Sample data script
├── src/main/java/com/cd/platform/
│   ├── CdPlatformApplication.java           ← Entry point
(@SpringBootApplication)
│   ├── PageController.java                  ← Thymeleaf page routing
│   ├── api/                                 ← REST controllers
│   ├── model/                               ← JPA entities + UserRole enum
```

```
│   ├── repository/                    ← JpaRepository interfaces
│   └── service/                       ← Business logic
└── src/main/resources/
    ├── application.yml                ← DB config, server port (8082)
    ├── templates/*.html               ← Thymeleaf pages (10 templates)
    └── static/assets/
        ├── css/styles.css             ← Custom design system
        ├── js/app.js                  ← Page logic & API fetch calls
        └── js/data.js                 ← Mock/fallback data
```

Chapter 4: Technologies Used

Category	Technology	Version	Role in Project
Backend Framework	Spring Boot	3.2.5	Application bootstrap, embedded Tomcat, auto-configuration
Language	Java	21 (LTS)	All backend source code
ORM / Persistence	Hibernate / Spring Data JPA	Jakarta EE	Entity mapping, CRUD repositories, schema validation
Database	MySQL	8.0	Relational persistence (cd_platform database)
View Layer	Thymeleaf	3.x	Server-side HTML template rendering
Validation	Jakarta Bean Validation	3.0	API request validation (@NotBlank, @Email, @NotNull)
Frontend JS	Vanilla JavaScript	ES2022	Dynamic UI rendering, fetch() API calls, fallback logic
CSS	Custom Design System	—	Premium UI: typography, colour system, components
Build Tool	Apache Maven (Wrapper)	3.8+	Dependency management, build lifecycle
IDE	Eclipse / IntelliJ IDEA	Any	Development environment

4.1 Key Dependencies (pom.xml)

- spring-boot-starter-web — REST controllers, embedded Tomcat server.
- spring-boot-starter-data-jpa — Hibernate, JPA repositories.
- spring-boot-starter-thymeleaf — Template engine for HTML pages.
- spring-boot-starter-validation — Jakarta Bean Validation for request DTOs.
- mysql-connector-j — JDBC driver for MySQL 8.0 (runtime scope).
- spring-boot-starter-test — Unit and integration testing support.

4.2 Application Configuration (application.yml)

```
spring.datasource.url: jdbc:mysql://localhost:3306/cd_platform
spring.datasource.username: root
spring.datasource.password: <your_password>
spring.jpa.hibernate.ddl-auto: validate
spring.jpa.show-sql: true
server.port: 8082
```

Note: ddl-auto=validate means Hibernate checks the schema matches entity definitions but does NOT create or alter tables. Tables must first be created manually or via seed.sql.

Chapter 5: Database Design

5.1 Entity-Relationship Overview

The MySQL database cd_platform contains eight tables that form a normalised relational schema. The central table is users, from which all domain entities derive their ownership. Developer and client profiles are one-to-one extensions of a user. Chat threads link users via a many-to-many join table (chat_participants).

Table	Key Columns	Relationships
users	id (PK), role (CLIENT|DEVELOPER), email (unique), password_hash, created_at	Parent of developer_profiles, client_profiles
developer_profiles	id (PK), user_id (FK→users), name, title, bio, skills, rate, location, availability, created_at	1:1 with users; 1:N with portfolio_projects
portfolio_projects	id (PK), developer_id (FK→developer_profiles), title, description, tech_stack, image_url	N:1 with developer_profiles
client_profiles	id (PK), user_id (FK→users), company_name, industry, budget_range, created_at	1:1 with users; 1:N with inquiries
inquiries	id (PK), client_id (FK→client_profiles), developer_id (FK→developer_profiles), project_summary, status, created_at	N:1 with client_profiles and developer_profiles
chat_threads	id (PK), updated_at	1:N with chat_participants, 1:N with chat_messages
chat_participants	thread_id (FK→chat_threads), user_id (FK→users) — composite PK	Join table: M:N between chat_threads and users
chat_messages	id (PK), thread_id (FK→chat_threads), sender_id (FK→users), content, created_at	N:1 with chat_threads and users

5.2 JPA Entity Summary

Each table has a corresponding @Entity class in com.cd.platform.model. Key annotations used:

- @Entity / @Table — maps class to table.
- @Id / @GeneratedValue(strategy = GenerationType.IDENTITY) — auto-increment primary key.
- @Column(nullable = false, unique = true) — enforces NOT NULL and UNIQUE constraints.
- @ManyToOne(fetch = FetchType.LAZY) + @JoinColumn — foreign-key relationships loaded lazily to avoid N+1 queries.
- @EmbeddedId / composite key for ChatParticipant (thread_id + user_id).

- @Enumerated(EnumType.STRING) — stores UserRole as 'CLIENT' or 'DEVELOPER' string in DB.
- @Column(insertable=false, updatable=false) — for created_at / updated_at columns managed by MySQL DEFAULT CURRENT_TIMESTAMP.

5.3 Database Setup & Seeding

To set up the database, run the following in MySQL Workbench or CLI:

```
CREATE DATABASE cd_platform CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE cd_platform;
-- Create tables (see AGENT.md Phase 1 DDL)
-- Then run the seed script:
SOURCE db/seed.sql;
```

The seed script inserts 4 users (2 clients, 2 developers), 2 developer profiles, 3 portfolio projects, 2 client profiles, 2 inquiries, 2 chat threads, and 4 chat messages — providing a complete data set for demo and testing.

Chapter 6: Module Description — Pages & Flows

6.1 Shared Pages

Landing Page (/) — index.html serves as the entry point. It features a bold hero section with a value proposition, a role-selection UI (Client or Developer), and a navigation to the login page. The page is rendered statically by PageController.

Login / Sign-Up Page (/login) — login.html provides two tabs: Login and Sign Up. The login form submits email and password to POST /api/auth/login. The sign-up form collects role, email, password, and role-specific fields (developer: name, title, bio, skills, rate, location, availability; client: company name, industry, budget range). On success, the response JSON (userId, role, developerId, clientId) is stored in localStorage and the user is routed to the appropriate home page.

6.2 Client Flow Pages

Client Dashboard (/client/home) — client-home.html shows a personalised welcome, a grid of featured/recommended developers fetched from GET /api/developers, and a collapsible client profile form that calls POST /api/clients or PUT /api/clients/{id} to save profile data.

Browse Developers (/client/developers) — client-developers.html renders a searchable, filterable listing of all developers. Developer cards show name, title, skills, rate, location, and availability. Filters include skill, location, and price range. Data is fetched from GET /api/developers; falls back to mock data from data.js.

Developer Profile (/client/developer?id=X) — client-developer.html displays a full developer profile including bio, skills, hourly rate, availability, and a portfolio project grid. An inquiry form at the bottom calls POST /api/inquiries to send a project summary. Portfolio cards are fetched from GET /api/developers/{id}/portfolio.

Client Chat Inbox (/client/messages) — client-messages.html presents a two-pane chat UI. The left pane lists all chat threads for the logged-in client (fetched from GET /api/chat/threads?userId={id}). The right pane shows the selected thread's messages (GET /api/chat/threads/{threadId}/messages) and a composer that calls POST /api/chat/threads/{threadId}/messages.

6.3 Developer Flow Pages

Developer Dashboard (/developer/home) — developer-home.html shows a profile completion status widget, a leads/inquiries summary panel listing incoming client project inquiries, and quick-action links to profile and portfolio management.

Developer Profile Editor (/developer/profile) — developer-profile.html is a form-based editor for all developer profile fields (name, title, bio, skills, hourly rate, location, availability). The save button calls PUT /api/developers/{id} with the updated data.

Portfolio Management (/developer/portfolio) — developer-portfolio.html displays all portfolio projects for the logged-in developer in a card grid (fetched from GET

/api/developers/{id}/portfolio). An 'Add Project' form at the bottom calls POST /api/developers/{id}/portfolio to add a new project entry.

Developer Chat Inbox (/developer/messages) — developer-messages.html mirrors the client chat UI, showing threads where the logged-in developer is a participant and enabling two-way messaging via the chat API.

Chapter 7: Backend API Reference

7.1 Authentication API — /api/auth

Method	Endpoint	Request Body	Response
POST	/api/auth/login	{email, password}	{userId, role, email, developerId, clientId}
POST	/api/auth/signup	{role, email, password, name/title/bio/skills/rate/location/availability OR companyName/industry/budgetRange}	{userId, role, email, developerId, clientId}

7.2 Developer API — /api/developers

Method	Endpoint	Request Body	Response
GET	/api/developers	—	List of DeveloperDto
GET	/api/developers/{id}	—	DeveloperDto or 404
POST	/api/developers	{userId, name, title, bio, skills, rate, location, availability}	DeveloperDto
PUT	/api/developers/{id}	{name, title, bio, skills, rate, location, availability}	Updated DeveloperDto
GET	/api/developers/{id}/portfolio	—	List of PortfolioDto
POST	/api/developers/{id}/portfolio	{title, description, techStack, imageUrl}	PortfolioDto

7.3 Client API — /api/clients

Method	Endpoint	Request Body	Response
GET	/api/clients/{id}	—	ClientDto or 404
POST	/api/clients	{userId, companyName, industry, budgetRange}	ClientDto
PUT	/api/clients/{id}	{companyName, industry, budgetRange}	Updated ClientDto

7.4 Inquiry API — /api/inquiries

Method	Endpoint	Request Body	Response
POST	/api/inquiries	{clientId, developerId,	InquiryDto

Method	Endpoint	Request Body	Response
		projectSummary, status}	

7.5 Chat API — /api/chat

Method	Endpoint	Request Body / Param	Response
GET	/api/chat/threads	?userId={id}	List of ChatThreadDto (with participantIds)
POST	/api/chat/threads	{userIds: [...]}	New ChatThreadDto
GET	/api/chat/threads/{threadId}/ messages	—	List of ChatMessageDto
POST	/api/chat/threads/{threadId}/ messages	{senderId, content}	ChatMessageDto

Chapter 8: Implementation Highlights

8.1 Two-Phase Development Approach

The project was built in two clearly defined phases. Phase 1 was a UI-first demo phase: complete frontend pages and flows were built using static mock data from data.js, with no real backend. This allowed rapid UI iteration and a working demo without any infrastructure dependencies. Phase 2 replaced mock data with real API calls and introduced the full Spring Boot backend, MySQL schema, and persistent data.

This approach proved highly effective: the UI was production-ready before a single API was written, and the clean API boundary in the JS layer (a single fetchDevelopers(), fetchPortfolio() abstraction) meant the switch from mock to real data required minimal changes to the frontend code.

8.2 AuthService — Transactional Signup Flow

The AuthService.signup() method is annotated @Transactional and creates a User, then creates either a DeveloperProfile (with optional first PortfolioProject) or a ClientProfile in a single atomic transaction. If any step fails, all database writes are rolled back. This prevents orphaned users without profiles.

8.3 Chat System Design

The chat system uses a standard threaded messaging model. A ChatThread represents a conversation; ChatParticipant is a join table that maps users to threads (many-to-many); ChatMessage belongs to a thread and has a sender. The ChatService.getThreadsForUser() method queries threads by user participation, and getMessagesForThread() returns all messages ordered by creation time. The API supports creating new threads (POST /api/chat/threads with a list of userIds) and sending messages (POST with senderId and content).

8.4 Frontend Fallback Pattern

The JavaScript in app.js uses a try/catch fetch-first pattern for every data call. If the API is unreachable or returns an empty result, the UI silently falls back to the rich mock data set in data.js. This makes the application demo-safe: it works with or without a running backend, which was critical for the Phase 1 demo milestones.

8.5 DTO Pattern in REST Controllers

All REST controllers use Java Records as DTOs (DeveloperDto, ClientDto, ChatMessageDto, etc.) rather than returning entity objects directly. This decouples the API contract from the JPA entity structure, prevents accidental serialisation of lazy-loaded relations (which would trigger LazyInitializationException), and keeps the API response shape stable even if the entity model changes.

Chapter 9: Testing

9.1 Manual Test Cases

#	Test Case	Input / Action	Expected Result	Status
1	Developer Sign-Up	POST /api/auth/signup with role=DEVELOPER, email, password, name, skills	201 OK; user + developer profile created; developerId returned	**PASS**
2	Client Sign-Up	POST /api/auth/signup with role=CLIENT, companyName, industry, budgetRange	201 OK; user + client profile created; clientId returned	**PASS**
3	Login — valid credentials	POST /api/auth/login with correct email + password	200 OK; correct role and IDs returned	**PASS**
4	Login — wrong password	POST /api/auth/login with wrong password	400 Bad Request / error message	**PASS**
5	Browse Developers	GET /api/developers	List of all developer profiles returned as JSON	**PASS**
6	Get Single Developer	GET /api/developers/1	DeveloperDto for id=1 returned	**PASS**
7	Update Developer Profile	PUT /api/developers/1 with updated rate and bio	Updated DeveloperDto returned; DB row updated	**PASS**
8	Add Portfolio Project	POST /api/developers/1/portfolio with title, techStack	PortfolioDto returned; project linked to developer	**PASS**
9	Create Inquiry	POST /api/inquiries with clientId=1, developerId=2, projectSummary	InquiryDto created; visible in seed data	**PASS**
10	Start Chat Thread	POST /api/chat/threads with userIds=[1,2]	New ChatThreadDto with id and participantIds	**PASS**
11	Send Chat Message	POST /api/chat/threads/1/messages with senderId=1, content	ChatMessageDto with correct threadId and content	**PASS**

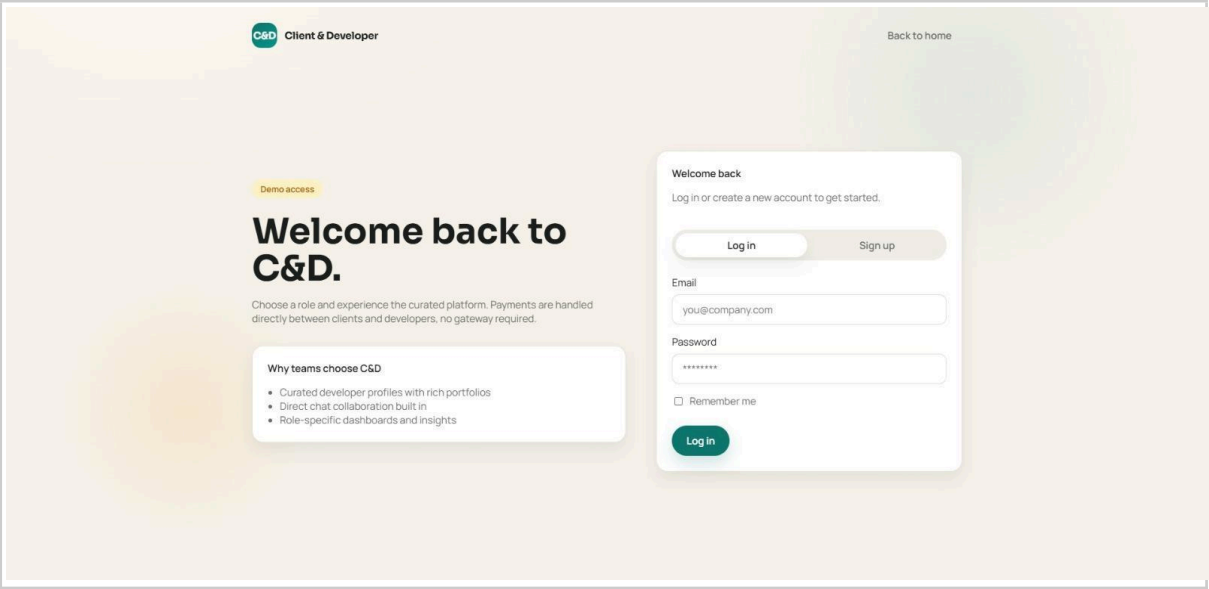
#	Test Case	Input / Action	Expected Result	Status
12	Get Thread Messages	GET /api/chat/threads/1/messages	Ordered list of all messages in thread 1	**PASS**
13	Client Profile Update	PUT /api/clients/1 with new budgetRange	Updated ClientDto; DB persisted	**PASS**
14	Frontend Fallback	Stop backend; navigate to /client/developers	Page loads with mock developer data from data.js	**PASS**

Chapter 10: Screenshots

This chapter provides visual documentation of the C&D platform's key pages and user flows, captured from the running application.

10.1 Authentication

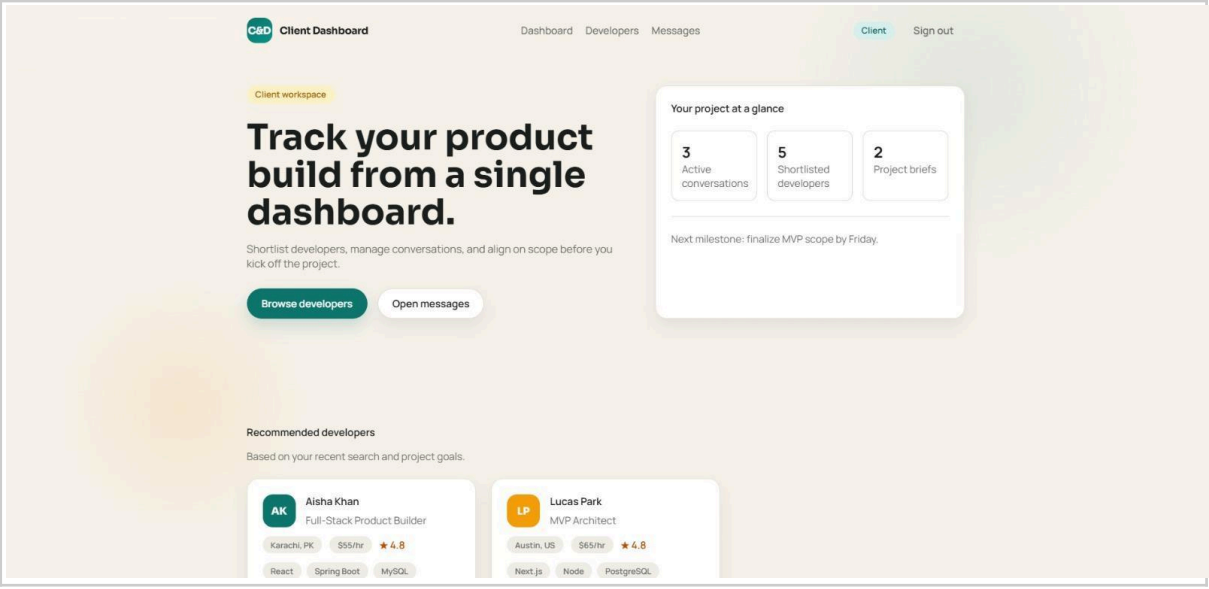
Fig 10.1 — Login Page (/login)



Login page with tabbed Login / Sign Up interface, email and password fields, and the 'Why teams choose C&D' feature panel.

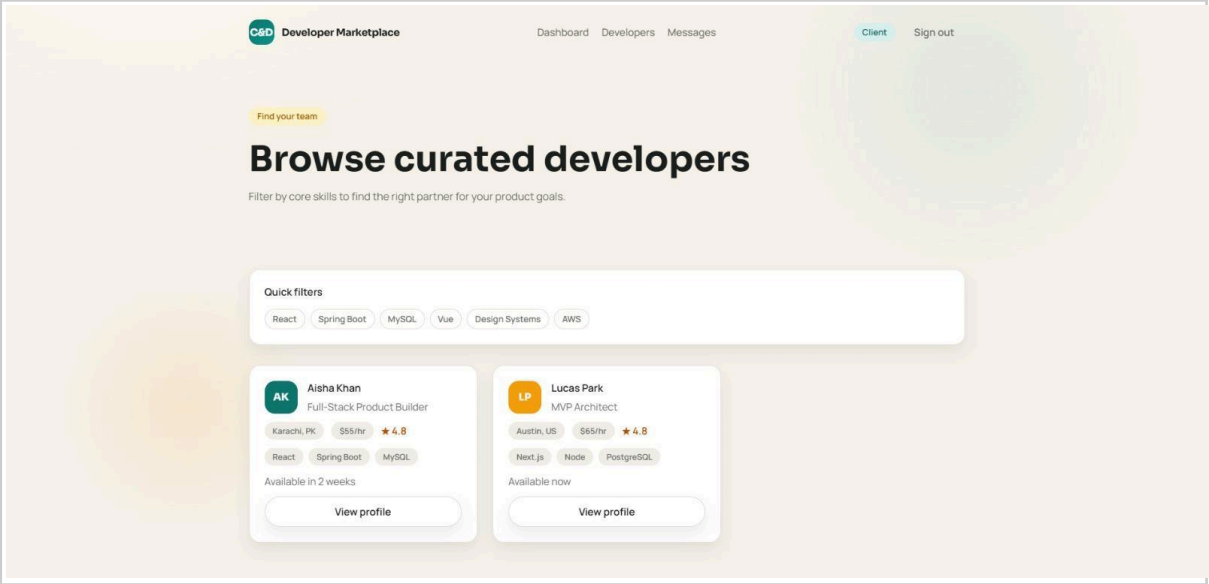
10.2 Client Flow

Fig 10.2 — Client Dashboard (/client/home)



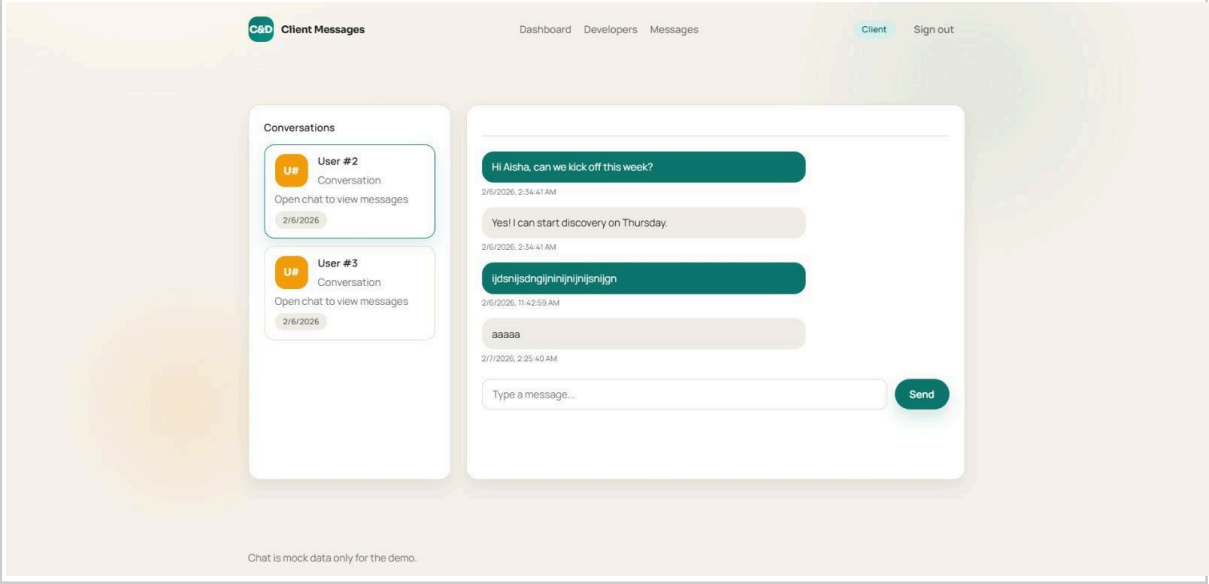
Client workspace dashboard showing project-at-a-glance stats (3 active conversations, 5 shortlisted developers, 2 project briefs), CTAs, and recommended developer cards.

Fig 10.3 — Browse Developers (/client/developers)



Developer Marketplace listing page with quick-filter skill tags (React, Spring Boot, MySQL, Vue, etc.) and developer cards showing name, title, rate, rating, and availability.

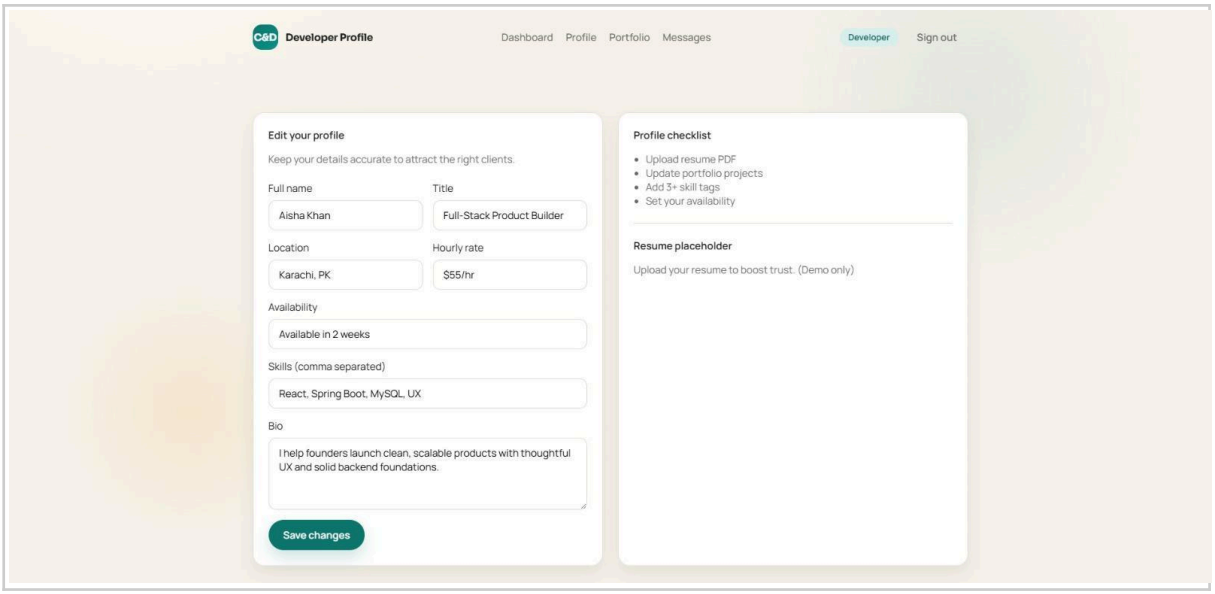
Fig 10.4 — Client Chat Inbox (/client/messages)



Two-pane messaging UI: conversation thread list on the left with timestamps, and the active chat on the right with alternating teal/grey message bubbles and a message composer.

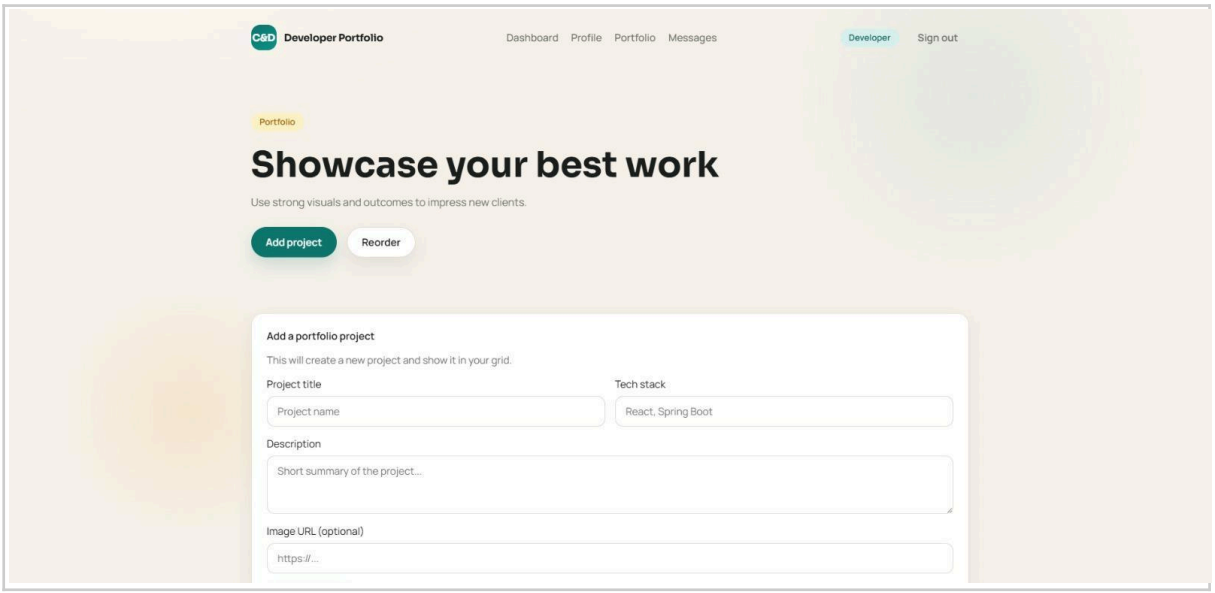
10.3 Developer Flow

Fig 10.5 — Developer Profile Editor (/developer/profile)



Developer profile editor with fields for full name, title, location, hourly rate, availability, skills (comma-separated), and bio — alongside a profile checklist and resume upload placeholder.

Fig 10.6 — Developer Portfolio (/developer/portfolio)



Portfolio management page with 'Showcase your best work' hero, Add Project and Reorder CTAs, and the Add Portfolio Project form with project title, tech stack, description, and image URL fields.

Chapter 11: Conclusion & Future Scope

10.1 Conclusion

The C&D Platform successfully delivers a complete, functional two-sided marketplace connecting clients and developers. The project demonstrates end-to-end full-stack development with Spring Boot (Java 21), Hibernate/JPA, MySQL, Thymeleaf, and a custom JavaScript frontend — all core technologies of the PG-DAC curriculum.

Key achievements include: a well-normalised 8-table MySQL schema, 16+ REST API endpoints covering all business domains, a role-based dual-flow UI with 10 Thymeleaf pages, a two-phase development methodology (mock-first, then real API), and a resilient frontend fallback pattern that ensures the application is always demo-ready.

The project fulfills all stated requirements and provides a solid, production-ready foundation for further development and feature expansion.

10.2 Future Scope

- Spring Security + JWT — Role-based access control, stateless session management, and protected API endpoints.
- OAuth2 / Google Login — Social login for frictionless onboarding.
- Real-time Chat with WebSocket — Replace the polling-based chat with Spring WebSocket (STOMP) for instant message delivery.
- File Upload — Resume PDFs and portfolio images stored on S3 or local filesystem.
- Search & Filter API — Backend-powered developer search by skill, location, and rate range.
- Ratings & Reviews — Clients can rate and review developers after a project.
- Payment Integration — Stripe or Razorpay for milestone-based project payments.
- Email Notifications — Spring Mail for inquiry alerts and new message notifications.
- Admin Dashboard — Platform admin panel for user management and analytics.
- Cloud Deployment — Docker + Docker Compose packaging; deployment on AWS EC2 or Railway.

References

1. Spring Boot Official Documentation — https://spring.io/projects/spring-boot

2. Spring Data JPA Reference — https://docs.spring.io/spring-data/jpa/docs/current/reference/html/

3. Hibernate ORM Documentation — https://hibernate.org/orm/documentation/

4. Thymeleaf Documentation — https://www.thymeleaf.org/documentation.html

5. Jakarta Bean Validation — https://beanvalidation.org/3.0/

6. MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/

7. Spring Boot Maven Plugin — https://docs.spring.io/spring-boot/docs/current/maven-plugin/reference/html/

8. Baeldung Spring Tutorials — https://www.baeldung.com/spring-boot

Appendix — Quick Start Guide

A. Prerequisites

- Java 21 (LTS) — verify with: java -version
- Maven 3.8+ or use the included Maven Wrapper (./mvnw)
- MySQL 8.0 — running locally with a root user

B. Setup Steps

```
# 1. Create database
mysql -u root -p -e "CREATE DATABASE cd_platform CHARACTER SET utf8mb4;"
# 2. Create tables (run DDL from AGENT.md Phase 1)
# 3. Seed sample data
mysql -u root -p cd_platform < db/seed.sql
# 4. Update password in application.yml
# 5. Run application
./mvnw spring-boot:run
# 6. Open browser
http://localhost:8082/
```

C. Page Routes Reference

URL	Page / Purpose
/	Landing page — role selection (Client or Developer)
/login	Login + Sign-Up page (tabbed)
/client/home	Client dashboard — featured developers + profile form
/client/developers	Browse all developers with search/filter
/client/developer?id=X	Individual developer profile + inquiry form
/client/messages	Client chat inbox — threads and messages
/developer/home	Developer dashboard — leads summary
/developer/profile	Developer profile editor
/developer/portfolio	Portfolio management — view + add projects
/developer/messages	Developer chat inbox